C++ Hackathon Design Notes

GROUP C11

Location: Kolkata

Team members

Aaditya Kumar Choudhary (ACHOUDH4)

Jitendra Singh (JSINGH15)

Sneha Lahiri (SLAHIRI)

Snigdhendu Chattopadhyay (SCHATTOP)

Tamanna Parween (TPARWEEN)

Contents

1 Project Overview

1.1 Introduction

The Smart Cabin & Vehicle Health Monitoring System is a production-grade, multi-threaded C++ application that simulates and monitors a vehicle's intelligent cockpit domain controller. It is implemented entirely in C++17 without any external libraries, adhering strictly to modern C++ best practices: RAII, smart pointers, concurrent execution, and STL-centric design.

The system is architected with an Adaptive AUTOSAR mindset — featuring service-oriented modules, thread-safe shared data, and event-driven alerting — making it directly analogous to real-world automotive ECU software.

1.2 Scope and Goals

- Simulate six real-world vehicle sensors with periodic data generation.
- Detect abnormal vehicle conditions and classify alerts by severity.
- Present a live, auto-refreshing ASCII dashboard in the terminal.
- Log all critical events asynchronously to a persistent file.
- Manage all concurrent activities safely using std::thread and std::mutex.
- Provide a manual input debug mode for controlled testing.

1.3 Real Automotive Mapping

System Component	Automotive Equivalent
Sensor Framework	MCAL / Sensor Abstraction Layer
Alert Manager	Vehicle Health Manager (VHM)
Dashboard	Instrument Cluster / HMI
Event Logger	Diagnostic Event Manager (DEM)
Thread Manager	Adaptive AUTOSAR Execution Context
Shared Data Guard	Inter-process Synchronization
Runtime Alerts	ISO 26262 Safety Monitoring

2 System Architecture

2.1 High-Level Architecture Diagram

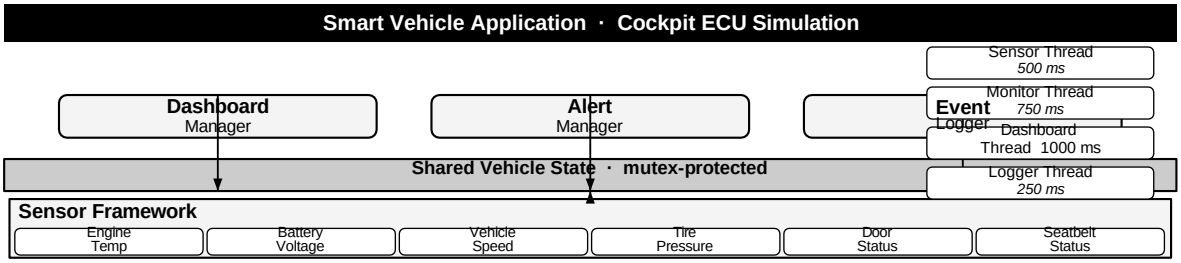

Figure 2.1: High-Level Software Architecture

2.2 Module Dependency Graph

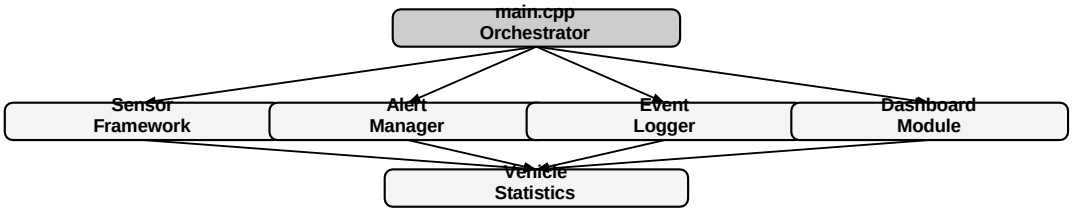

Figure 2.2: Module Dependency Graph

3 Sensor Framework

3.1 Class Hierarchy

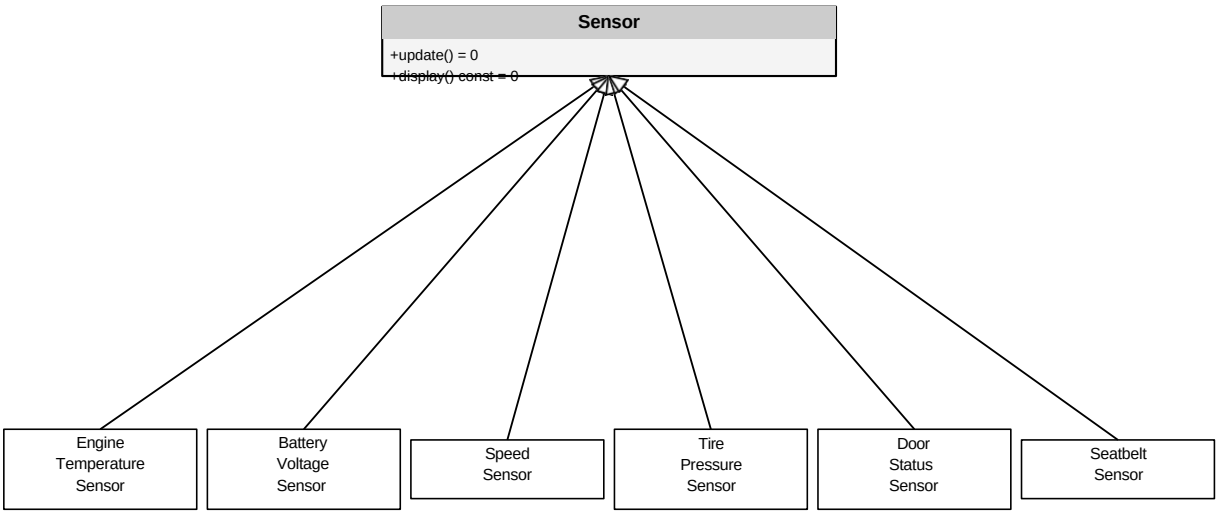

Figure 3.1: Sensor Class Hierarchy (UML Inheritance Tree)

3.2 Sensor Data Ranges and Alert Thresholds

Sensor	Range	Unit	Alert Threshold
Engine Temperature	70 – 120	°C	> 110°C CRITICAL
Battery Voltage	9.5 – 14.8	V	< 10 V WARNING
Vehicle Speed	0 – 220	km/h	> 120 km/h WARNING
Tire Pressure	20 – 40	PSI	< 25 PSI WARNING
Door Status	Open/Closed	—	Open + Speed > 10 km/h CRITICAL
Seatbelt	Locked/Unlocked	—	Unlocked while moving WARNING

3.3 Sensor State Machine

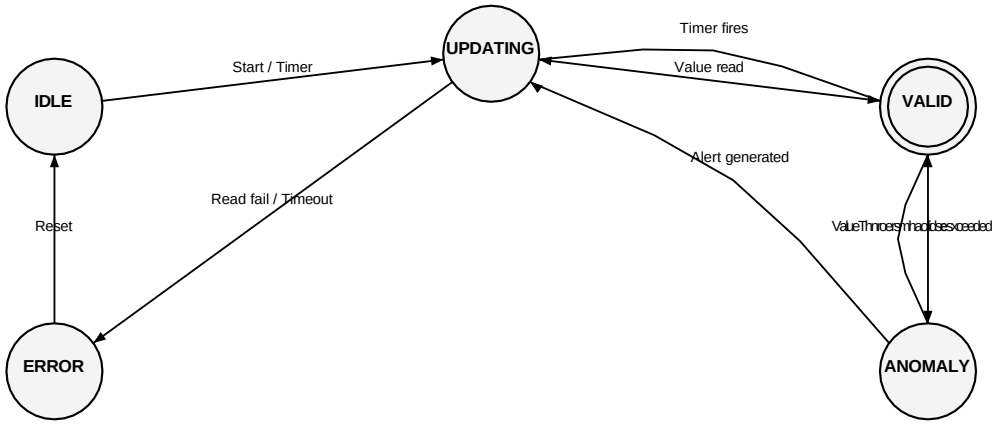

Figure 3.2: Sensor Lifecycle State Machine

3.4 Sensor Update Flowchart

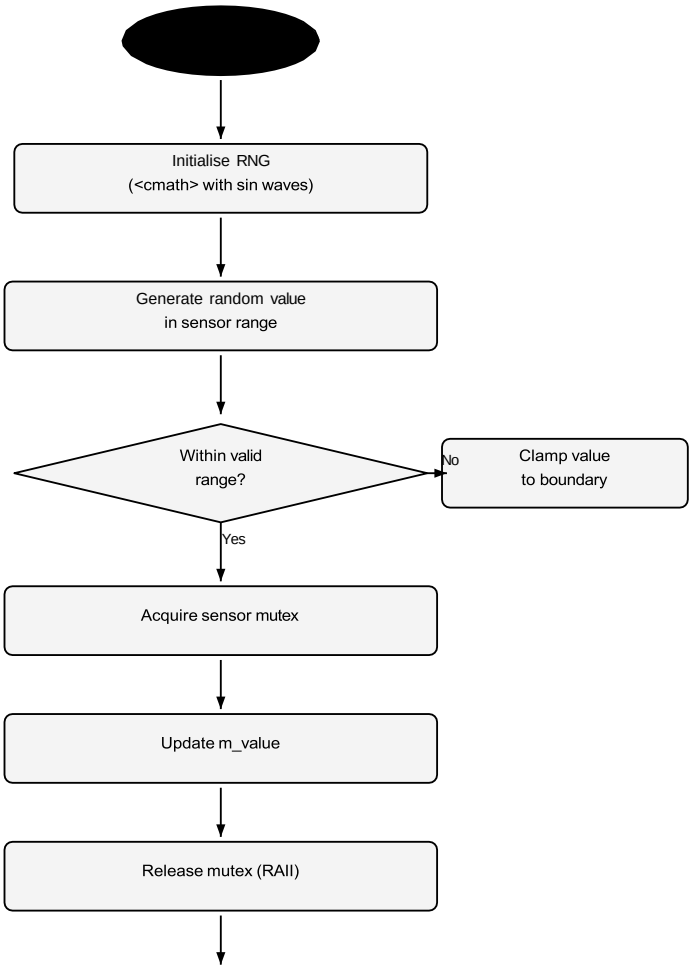

Figure 3.3: Sensor Update Thread Flowchart

4 Alert Management Module

4.1 Alert Severity Classification

Severity	Enum Value	Conditions
INFO	AlertSeverity::INFO	Informational, non-critical events
WARNING	AlertSeverity::WARNING	Battery low, overspeed, tire pressure, seatbelt
CRITICAL	AlertSeverity::CRITICAL	Engine overheat, door open while moving

4.2 Alert Lifecycle State Machine

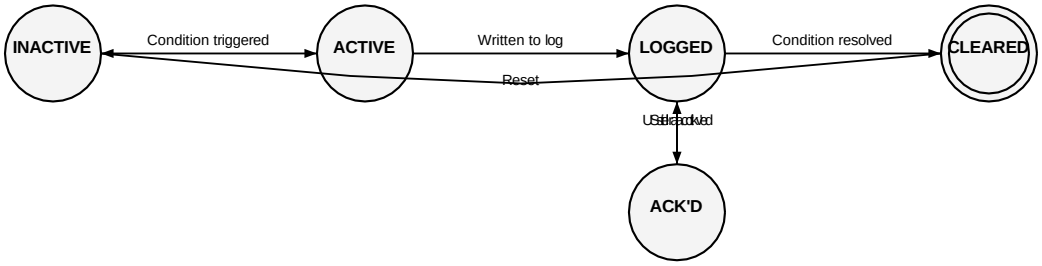

Figure 4.1: Alert Lifecycle State Machine

4.3 Alert Evaluation Flowchart

Monitor Thread Wakes
Read all sensor values (locked)
Engine Temp > 110°C? → CRITICAL: ENGINE OVERHEAT
Battery < 10V? → WARNING: LOW BATTERY
Speed > 120 km/h? → WARNING: OVERSPEED
Tire Pressure < 25 PSI? → WARNING: LOW TIRE PRESSURE
Door Open & Speed > 10? → CRITICAL: DOOR OPEN
Seatbelt OFF while moving? → WARNING: SEATBELT
Push alert(s) to queue (locked)
SLEEP 750 ms

Figure 4.2: Alert Evaluation Flowchart (Monitor Thread)

5 Event Logger Module

5.1 Async Logging Architecture

The Event Logger employs a producer-consumer model using a thread-safe queue. This decouples alert generation from disk I/O, preventing monitor-thread stalls.

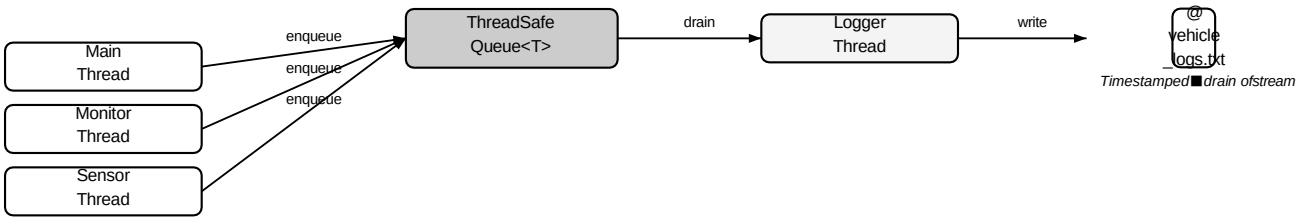

Figure 5.1: Async Event Logger Architecture (Producer-Consumer Pipeline)

5.2 Logger State Machine

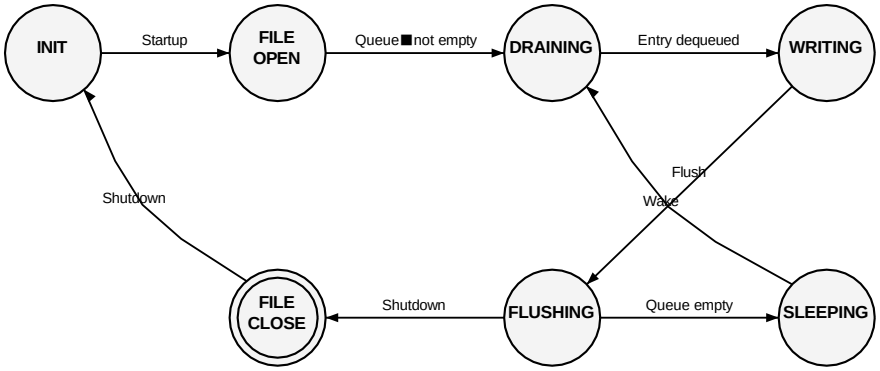

Figure 5.2: Event Logger State Machine

5.3 Log Entry Format

```
[2025-06-01 14:32:01] [CRITICAL] ENGINE OVERHEAT | EngineTemp=115.3 C
[2025-06-01 14:32:03] [WARNING]  LOW BATTERY | BatteryV=9.8 V
[2025-06-01 14:32:07] [WARNING]  OVERSPEED | Speed=135.2 km/h
[2025-06-01 14:32:09] [CRITICAL] DOOR OPEN WARNING | Door=OPEN Speed=22.1 km/h
[2025-06-01 14:32:11] [INFO]     System healthy | All sensors nominal
```

Listing 5.1: Sample log output in vehicle_logs.txt

6 Threading & Synchronization

6.1 Thread Execution Overview

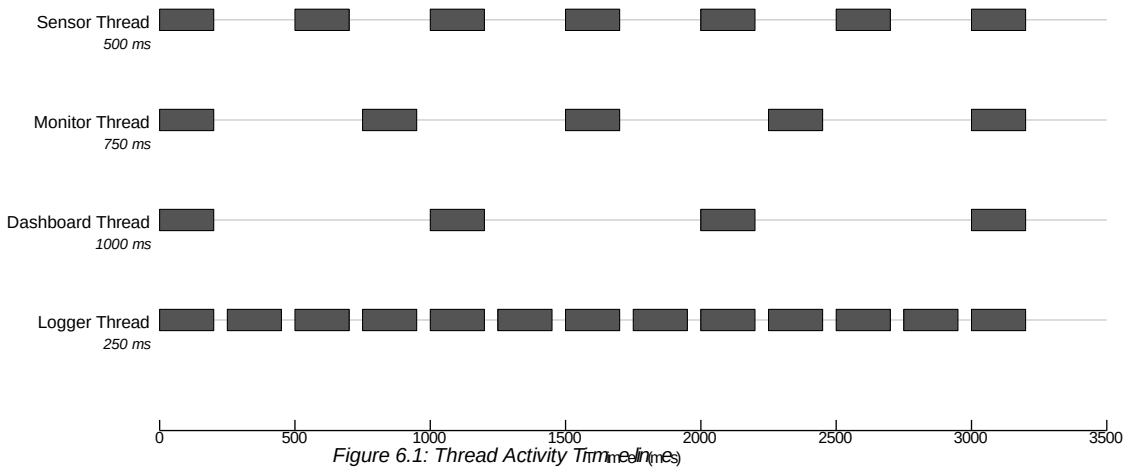

Figure 6.1: Thread Activity Timeline

6.2 Thread Synchronization Model

Shared Resource	Readers	Writers	Guard Mechanism
Sensor values	Monitor, Dashboard	Sensor	Per-sensor std::mutex
Active alert list	Dashboard, Logger	Monitor	std::mutex + lock_guard
Alert history	Dashboard	Monitor	Same alert mutex
Log queue	Logger thread	All	ThreadSafeQueue internal mutex
Statistics map	Dashboard	Sensor	Statistics mutex
Config flags	All	Main	Atomic read, start-up only

6.3 Startup and Shutdown Flowchart

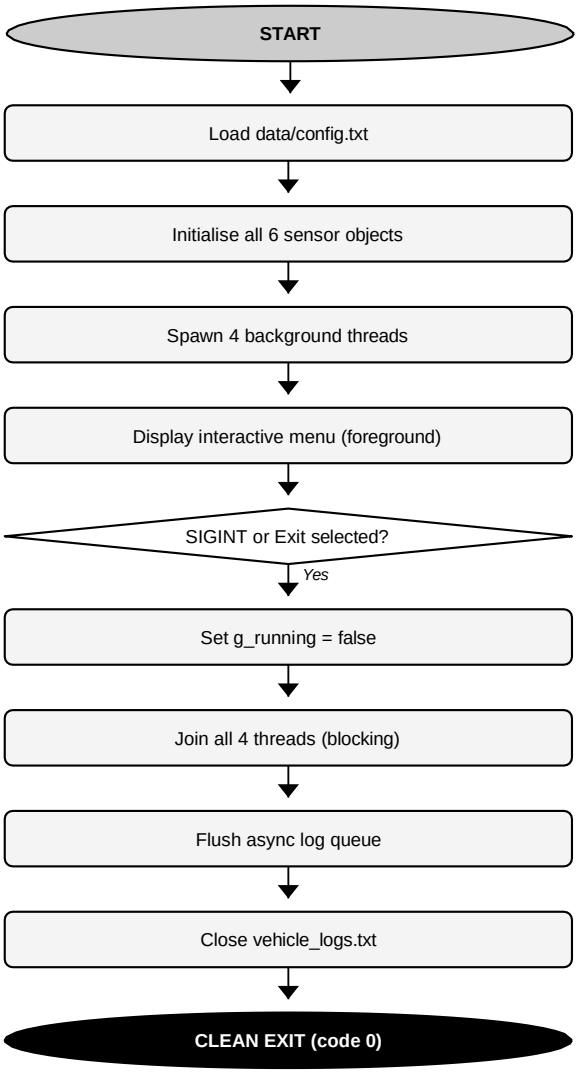

Figure 6.2: Application Startup and Graceful Shutdown Flowchart

7 Dashboard Module

7.1 Dashboard State Machine

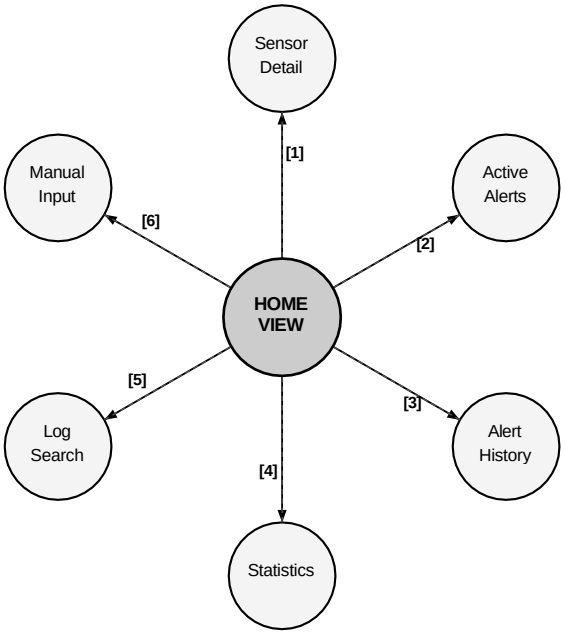

Figure 7.1: Dashboard Navigation State Machine

7.2 Dashboard Refresh Flowchart

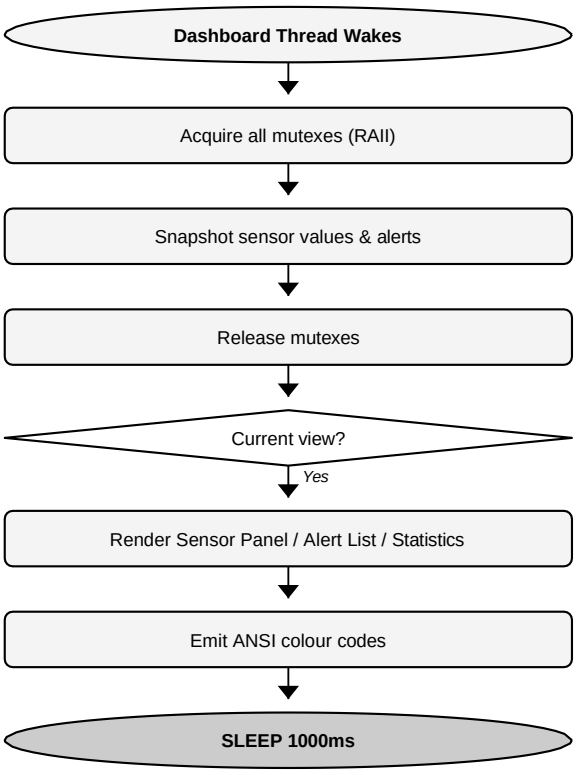

Figure 7.2: Dashboard Refresh Flowchart

8 Modern C++ Concepts Used

8.1 Concept Coverage Matrix

C++ Feature	Where Used	Why
Abstract Base Class / Virtual Functions	Sensor base class	Runtime polymorphism; dispatch to correct update()
RAII	File handles, mutex locks	Guaranteed release; no leaks on exceptions
unique_ptr / shared_ptr	Sensor objects, alert entries	Safe ownership, no raw new/delete
std::thread + std::mutex	All 4 background threads	Concurrent sensor updates without data races
std::lock_guard	Every shared data access	Automatic unlock; exception-safe
enum class	AlertSeverity	Scoped, type-safe severity levels
Operator Overloading (<<)	Alert class	Formatted output via stream operators
Lambda Expressions	Log search, alert filtering	Concise predicates for std::copy_if
std::vector, std::deque, std::map	Alert list, history, statistics	Chosen per access-pattern requirements
Templates	ThreadSafeQueue<T>	Generic, reusable concurrent queue
std::mt19937	Per-sensor RNG	High-quality pseudo-random sensor simulation
Move Semantics	Alert event objects	Efficient transfer without unnecessary copies
Static Members	Alert counter, object tracking	Class-level bookkeeping
Exception Handling	File open failures, bad data	Robust error handling without crashes

9 Build, Configuration & Usage

9.1 Folder Structure

```
cpp_hack/
  include/
    Sensor.hpp       # Abstract base + derived sensor headers
    Alert.hpp        # Alert class and AlertManager
    Logger.hpp       # ThreadSafeQueue and EventLogger
    Dashboard.hpp    # Dashboard and VehicleStatistics
  src/
    main.cpp         # Orchestration, thread launch, SIGINT handler
    Sensor.cpp
    Alert.cpp
    Logger.cpp
    Dashboard.cpp
  data/
    config.txt       # Runtime configuration flags
  logs/
    vehicle_logs.txt # Timestamped event log (auto-created)
  tests/
    test_cases.py
    test_runner.py
    test_report.md
  Makefile
  slog               # Bash smart log viewer
  README.md
```

9.2 Build Instructions

```
# Clean previous build artifacts
make clean

# Compile with C++17 (GCC / Clang required)
make

# Run the application
./vehicle_monitor
```

9.3 Configuration Reference (data/config.txt)

Key	Default	Description
MANUAL_INPUT_MODE	0	Set to 1 to enable manual sensor value injection
SENSOR_INTERVAL_MS	500	Milliseconds between sensor updates
LOG_FILE_PATH	logs/vehicle_logs.txt	Output log file path
MAX_ALERT_HISTORY	100	Maximum entries retained in alert history deque
OVERSPEED_LIMIT	120	Speed threshold (km/h) for overspeed alert
ENGINE_TEMP_LIMIT	110	Temperature threshold (°C) for engine alert
BATTERY_MIN_VOLT	10.0	Voltage floor (V) for low battery alert
TIRE_MIN_PSI	25	Pressure floor (PSI) for tire alert

9.4 Smart Log Viewer (slog)

```
chmod +x slog

./slog              # Print all log entries
./slog -f           # Tail the log in real-time (tail -f mode)
./slog -n 30        # Show the last 30 log entries
./slog CRITICAL     # Filter entries by severity level
./slog "ENGINE OVERHEAT"  # Search for a specific alert phrase
```

```
./slog --clear      # Wipe the log file
```

10 Testing Strategy

10.1 Test Coverage Overview

Test Area	Approach	Tool
Sensor range clamping	Regression test; feed boundary values	tests/ + assert
Alert condition evaluation	Regression test; mock sensor readings	Manual + MANUAL_INPUT_MODE
Thread safety	Race condition detection	Valgrind / ThreadSanitizer
Log file persistence	Verify entries after clean restart	slog script
Graceful shutdown	SIGINT; verify all threads joined	OS signal + exit code 0
Memory leak detection	Full lifecycle run	Valgrind --leak-check=full

10.2 Manual Test Scenarios Using Debug Mode

1. Set MANUAL_INPUT_MODE=1 in config.txt.
2. Launch ./vehicle_monitor.
3. Choose [6] Manual Sensor Input.
4. Enter engine temperature of 115°C → verify CRITICAL: ENGINE OVERHEAT appears.
5. Enter speed 130 km/h and door = OPEN → verify both overspeed and door-open alerts.
6. Enter battery voltage 9.5 V → verify WARNING: LOW BATTERY.
7. Restore values to nominal → verify alerts clear from active list.
8. Press Ctrl-C → verify clean exit with no orphan threads.

11 Automotive Context & Learning Outcomes

11.1 Mapping to Adaptive AUTOSAR Concepts

Hackathon Concept	Adaptive AUTOSAR / Industry Equivalent
Sensor abstract base	AUTOSAR Sensor & Actuator Abstraction
AlertManager	Vehicle Health Manager (VHM)
EventLogger	Diagnostic Event Manager (DEM)
Dashboard	HMI / Instrument Cluster IPC
std::thread	Adaptive AUTOSAR Execution Context
std::mutex	Inter-process / inter-runnable synchronisation
g_running atomic flag	Watchdog / lifecycle management
RAII file handles	Resource Manager in safety-critical software
ThreadSafeQueue	AUTOSAR Ara::Com service queuing

A Key Code Snippets

A.1 Abstract Sensor Interface

```
class Sensor {
public:
    virtual void update() = 0;
    virtual void display() const = 0;
    virtual std::string getName() const = 0;
    virtual ~Sensor() = default;

protected:
    mutable std::mutex m_mutex;
    std::mt19937 m_rng{ std::random_device{}() };
};
```

Listing A.1: Sensor base class (include/Sensor.hpp)

A.2 Thread-Safe Queue Template

```
template
class ThreadSafeQueue {
public:
    void push(T item) {
        std::lock_guard lock(m_mutex);
        m_queue.push_back(std::move(item));
    }
    bool pop(T& out) {
        std::lock_guard lock(m_mutex);
        if (m_queue.empty()) return false;
        out = std::move(m_queue.front());
        m_queue.pop_front();
        return true;
    }
private:
    std::deque m_queue;
    std::mutex m_mutex;
};
```

Listing A.2: Generic lock-based producer-consumer queue

A.3 Alert Evaluation with Lambda Filtering

```
auto results = std::vector{};
std::copy_if(
    m_history.begin(), m_history.end(),
    std::back_inserter(results),
    [&keyword;](const LogEntry& e) {
        return e.message.find(keyword) != std::string::npos;
    }
);
```

Listing A.3: Lambda-based log search in EventLogger

A.4 RAII Mutex Guard Pattern

```
void MonitorThread::evaluate() {
    // Snapshot under lock; release before alert processing
    SensorSnapshot snap;
    {
        std::lock_guard lock(m_sensorMutex);
        snap = collectSnapshot();
    } // mutex released here automatically

    if (snap.engineTemp > 110.0f)
        m_alertManager.raise(AlertSeverity::CRITICAL, "ENGINE OVERHEAT");
}
```

Listing A.4: RAII lock usage in monitor thread